Pricing a developed and an emerging insurance market

IAA AI-Assisted Hackathon 2026: final write-up

Actuarial Noob (Krishna Kumar Shrestha)

28 October 2026

	Week 1: developed market	Week 2: emerging market
Data	2021-24, 163,150 claims; 626,639 rows to price	2023-24, 9,258 claims; 209,063 rows to price
Submitted file	geometric mean of six experts, recalibrated per line	v13: tested ensemble ranking, segment levels set for the 2025 mix
Public / private D^2	0.12851 / **0.13830**	**0.17172** / pending (v12, public 0.17095, also selected)

Both weeks ask for the expected 2025 claim cost of every policy, scored by the Tweedie deviance D^2 with power 1.5 over all three lines. A D^2 of 0 is what you get by predicting the average for everyone, so every number below is a gain over that baseline. I ran both weeks the same way: get the target right first, test every idea on a year the model has not seen, and keep only what holds up. In the new market I started from what the developed market already knew and let the local data correct it.

1 Modelling approach

1.1 Data and target

Both weeks go through the same scripts (read, clean, build features), each with an executed notebook as evidence. Every fix is logged with the rows it changed. I dropped no rows and imputed nothing. The audit found customer ids that had lost their leading zeros (123,664 rows), exposure recorded in days or on a 360-day basis (13,880 and 595 rows in the two weeks), sums insured entered in units instead of thousands (3,527 and 349), a case reserve typed with an extra digit (11.9m instead of 1.19m), and impossible years licensed, restored from the same policy's other years (3,717 rows). The last one found was 604 claims stored twice under a second id, which had inflated 2022 claim counts by 1.4%.

The target is the ultimate cost of the claims with a loss date inside the row's period, net of excess and recoveries, with repudiated claims left out. Claims are valued at 31 December 2024, so 2024 is not fully reported: only about half of its bodily-injury cost is in. Every model's offset is log(exposure × share of claims reported), with the share taken from a reporting pattern per peril. Open claims are developed with the recovery ratio seen on closed ones, costs are trended to 2025 per peril, and costs above the 99.5% quantile are capped and added back as a load per line.

1.2 How every idea was tested

No change went in on the strength of one number. Each one had to pass tests that copy how the model is used, which means fitting on earlier years and scoring a later one:

Test	Question it answers	Scale
Out-of-time back-tests	Does it work on a year it has not seen?	Week 1: fit 2021-22, score 2023; fit 2021-23, score 2024. Week 2: fit 2023, score 2024
Customer-grouped 5-fold cross-validation	The same in Week 2, where one forward year is too little	every expert, setting and blend, 2023 and 2024 folds
Random “probe” columns	Does a candidate feature carry signal or luck?	278 candidate columns in Week 1 and 270 in Week 2: rules first, then 20 probes per column
Settings and blend search	Which tree size, ridge and expert mix?	42 model settings; all 63 expert combinations per line
Permutation test	Does a factor search find more than shuffled data would?	120 rating factors
Kaggle public score	How does a file score on part of 2025?	9 of the 15 Week 2 files I built

Rules drop leakage, governance columns and constants first. The probe pass mark then lets at most 1 in 20 noise columns through. Week 1 kept 146 of its 278 candidates; Week 2 kept 51 of 270, because two years of a small book cannot prove weaker effects. The features that survived make actuarial sense: a claim last year multiplies next year’s frequency by 1.7 to 2.1 in the developed market.

1.3 Week 1: six experts and a geometric mean

For each line I trained six experts on three fit sets (to price 2023, 2024 and 2025): a Tweedie GBM on total cost (the single-model baseline), a Tweedie GLM, one GBM per peril, and three frequency × claim-size models (Gamma claim size, lognormal claim size with a risk-specific variance, and frequency × claim size for each peril). Their predictions are combined row by row with an equal-weight geometric mean, then each line is recalibrated with a two-parameter Tweedie GLM, $\log E = \log(\text{exposure} \cdot L) + b_0 + b_1 \log(\text{geo}\ /L)$, where L is the line’s mean rate.

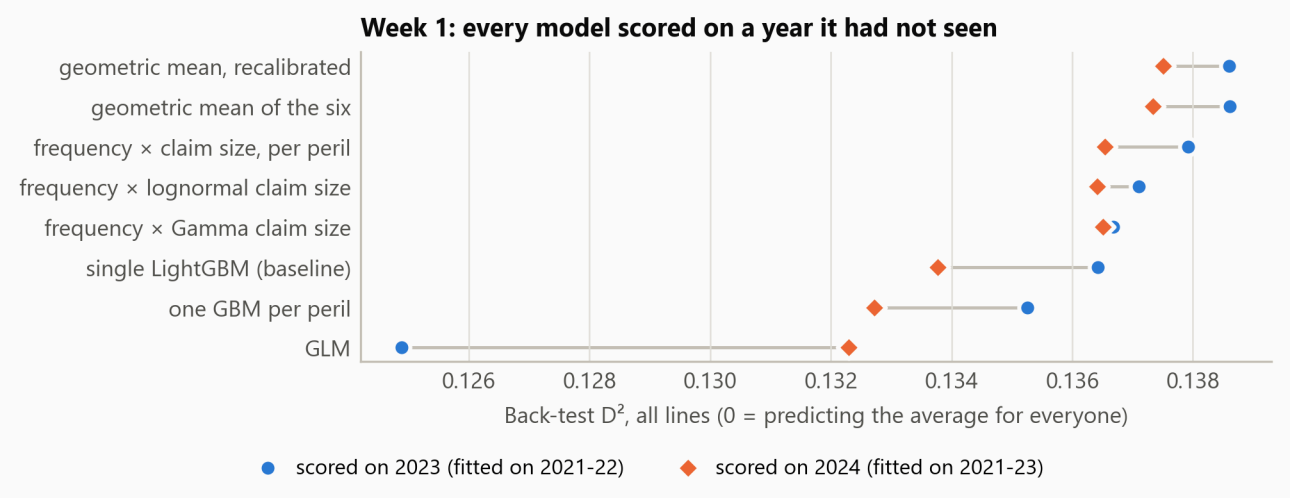


Figure 1: Week 1 back-tests, all lines pooled. The ensemble is best in both years; the recalibration was fitted on one year and scored on the other.

The ranking held across both years (Figure 1). The frequency × claim-size experts beat the single LightGBM in every line and year, and the geometric mean beat each expert: 0.1386 and 0.1375 against the baseline’s 0.1364 and 0.1338. The recalibration (b_1 = 1.04, 1.10 and 1.09 for motor, buildings and contents), fitted on one year and scored on the other, kept or slightly raised the score. A seventh expert gave the same private score, so I kept the simpler file. The figure comes from the rebuilt pipeline. The submitted file came from the 22 October code, which reported 0.1380 and 0.1367, and its 2025 predictions have a log correlation of 0.996 to 0.998 with the rebuild.

2 Adaptation to the new market

Week 2 differed from Week 1 in most of the ways that matter to a pricing model:

	Week 1	Week 2
History	4 years, 163k claims	2 years, 9k claims (about 1,000 per home line)
New business in the year priced	close to the training mix	77% of 2025, against 12-15% of training rows
Gender	not a rating factor	a permitted rating factor
Weather of the year	inside the training range	2025 far outside it (rainfall anomaly −128 mm against ±3 mm)

Start from the developed market. I fit a LightGBM on every Week 1 row using only the factors that mean the same thing in both markets (30 for motor, 24 for buildings, 26 for contents) and put its log rate into the offset of the Week 2 models. The local trees then only learn how this market differs. Figure 2 shows why: with two years of data alone, buildings reached a D^2 of only 0.012; on top of the prior it reached 0.075, and all lines together went from 0.103 to 0.128.

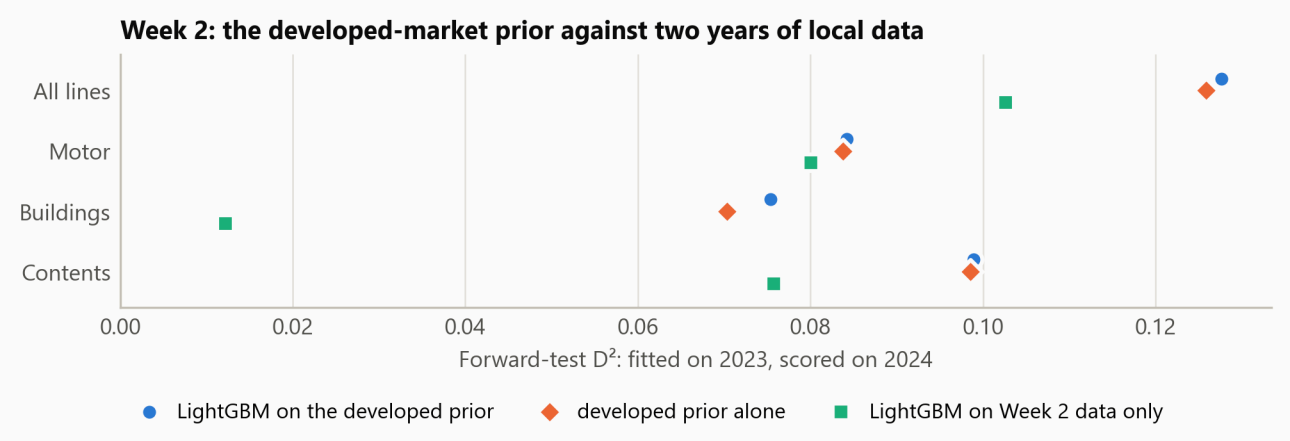


Figure 2: Week 2 forward test (fit on 2023, score 2024). The values are in appendix C.

Keep the local models simple. Cross-validation chose one-split trees for motor and contents and a strong GLM ridge; a weak ridge overfit buildings to a D^2 below zero. Searching all 63 expert combinations per line showed that all six experts together lost to fewer (contents: 0.1071 against 0.1087 for the total-cost model alone), because two years hold too few claims to learn claim sizes. Each line keeps its best set, which lifted the forward test from 0.1278 to 0.1300. A fitted recalibration slope changed the other year's D^2 by −0.0035 to +0.0008 with no steady sign, so the slope stays at 1. New-business and gender factors are measured on out-of-fold predictions and shrunk towards no effect. I dropped the weather of the year, since 2025 lies outside anything seen.

Where the model was wrong. The errors were mostly in levels. In the forward test buildings came in at 0.59 of the predicted cost, because 2023 had a fire-cost spike, and new business, which is only 13% of the 2024 rows, could not be checked at all. The ranking within a segment could be tested; the level of a segment under the 2025 mix could not.

Checking against Kaggle, and setting segment levels. Kaggle did two jobs. For model changes it was the last check: a new model went up only after it passed the tests above. The first such check showed that the ensemble (v4), although it ranked the 2024 risks better, scored below my earlier single transfer LightGBM (0.16198 against 0.16529), and the reason was the levels. So I kept the tested ranking inside each of ten segments (line × new business × gender) and set only the segment levels from outside evidence. v5 keeps the earlier file's segment totals with a 50/50 blend of rankings, which on 2024 rows reweighted to the 2025 mix ranks better (within-segment D^2 0.1067 against 0.1016). For the levels, the public score was the only evidence about 2025, which is its second job. Because it covers just part of 2025, I modelled its sampling noise (8% to 24% per segment) and built two files: v12 aims at the rows not yet scored and pulls noisy segments back, and v13 aims at the public rows (appendix B). v12 scored 0.17095 and v13 0.17172. I selected both, and v13 is the file submitted here. Ranking-only changes moved the public score less than its noise (v14, 0.0006 below v12 against noise of 0.0008), so every ranking decision stayed with cross-validation.

3 Major decisions

Decision	Rejected	Evidence
Fix data errors with evidence; drop and impute nothing	dropping or imputing rows	cleaning log per round
Keep a feature only if it beats random columns	keeping every plausible column	146 of 278 kept (Week 1), 51 of 270 (Week 2)
Geometric mean of experts, recalibrated per line	one LightGBM	+0.0022 (2023) and +0.0037 (2024) back-test D^2
Developed-market prior in the offset	Week 2-only models	+0.025 forward D^2
Simple local models, members per line by cross-validation	all six experts, fitted slope	contents: all six 0.1071 against `total` alone 0.1087
Ranking from tests, segment levels from 2025 evidence	trusting the back-test's levels	v4 0.16198 against v5 0.16529
Aim at unseen rows as well as the public sample	chasing the public score alone	public noise 8% to 24% per segment

4 Use of AI

I worked alone, with an AI coding agent (Claude Code in VS Code) at every step. It wrote and ran most of the code and drafted this write-up. I set the questions, checked the evidence and decided what was built.

Most of the work was a long run of prompts, each asking the agent to try one idea and test it: the developed-market prior, the expert ensembles, the member search, the permutation test. An idea counted only if it held up in the tests of section 1.2.

The AI model itself never changed during the project; its memory did. `KNOWLEDGE.md` records every idea tried, its test result and why it was kept or dropped, and the submission register does the same for every file. Each new task started by giving the agent that record and the question to answer, and each finished test was written back. This loop of trying, testing, writing it down and reusing it is the basic idea behind memory-based agent methods such as Memento (Zhou et al., 2025), done by hand: I chose which past results the agent saw, where Memento learns that choice. It kept the agent from starting over or repeating dead ends between sessions.

Two project skills in the repository (`.claude/skills/data-scientist` and `ml-engineer`) set how the agent works: frame the problem first, test out of time, distrust its own gains, and keep the code reproducible and under version control.

I did not accept a result from the same run that produced it. Each idea was questioned on its own, so the agent had to check its own work, and this caught real problems. Factor searches looked strong when the same two years both chose and scored the factors; the permutation test then showed real factors pass no more often than shuffled ones (motor 8 of 45 against 7.5 expected by chance), and only gender survived. A later review found a sign error in the Week 1 contents frequency trend, which is documented, left in the code that reproduces the submitted file, and fixed in the rebuild. Nothing went into the pipeline or a submission until I had reviewed the plan and its evidence.

5 Assumptions

- 2025 claims follow 2023-24 after trend (motor severity 0 to 7.9% a year depending on peril, buildings severity 6%, contents frequency 1.4% and severity 3.4%). Reporting patterns are stable, and open claims develop the way closed ones did.
- For factors defined the same way in both markets, the developed market's relativities are a reasonable starting point; the emerging market differs in level and in a few effects the local models can learn.
- The public leaderboard rows are a random sample of 2025, about 30%. The order of v12 and v13 holds for any share from 20% to 50%.
- Gender is a permitted rating factor in the emerging market, as the case study states.

6 Limitations

- Buildings is the part I trust least. There are only two years of emerging-market data, a few large claims decide its results, and its segment levels carry 8% to 10% uncertainty.
- The Week 2 segment levels rest partly on public scores. The competition allows that, but in real pricing it would mean calibrating on a sample of the book's own outcomes.
- The Week 2 expert sets were chosen on folds that include 2024, so their forward-test gain is a little optimistic.
- The submitted Week 1 file keeps the contents trend sign error. The per-line recalibration absorbs most of it (the intercept is about +10%), but contents relativities may be slightly off.

7 Guardrails

- Columns driven by the outcome are excluded: `renewal_decision`, and `customer_service_contacts`, which is zero on 0.2% of rows with a claim against 67% of rows without one. Claims history is built as known on 1 January of each year, so no feature looks ahead.
- The written premium is excluded, because a model using it would copy the old tariff. So are marketing and service fields and personal data such as surname. Gender is used only where permitted, and its factor moves price between men and women without changing a line's total.
- Every model change is tested out of time before its file is submitted. Every file is registered as a new numbered version with its evidence, description and checksum, and never overwritten.
- A file is saved only if it has one row per sample row id, in order, with every prediction finite and positive. The Week 1 code adds block and escalate rules: a back-test D^2 at or below zero blocks the file; an A/E outside 1 ± 5%, feature drift with a PSI of 0.25 or more, or a pure-premium shift above 15% against history goes to actuarial sign-off.
- Probe columns, permutation tests and the public-noise model are all there to stop me fitting noise, including apparent gains that an AI agent reports with confidence.
- Every step is a script under git, and `final/pipeline.py` runs them in order. From the intermediate results in the repository it rebuilds the Week 2 file byte for byte and the Week 1 file to a relative difference of 3e-15, in about a minute. A full rerun from the raw Week 2 data on a fresh clone rebuilt the Week 2 file to within 9e-7.

Appendix (outside the five graded pages)

A. Experiment log: every Week 2 file (from `2nd round/submissions/versions.csv`).

Version	Public D^2	What it is
v1	0.12519	starter: one LightGBM per line, no prior, no trend or large-loss treatment
v2	not scored	LightGBM on the developed prior, with new-business and gender factors
v3	not scored	ensemble on developed priors, recalibration slope fitted (kept for comparison)
v4	0.16198	ensemble on developed priors, level only (slope 1)
v5	0.16529	earlier best file's segment totals; ranking 50% that file, 50% ensemble
v6	not scored	as v5, ranking 25% / 75%
v7	not scored	probe: v5 with motor new business x1.15
v8	0.16552	probe: v5 with buildings new business x1.08
v9	not scored	probe: v5 with contents new business x1.10
v10	0.16556	probe: v5 with motor renewals x1.07
v11	0.17036	v5 with every segment at its best level from all scores
v12	0.17095	v5 with every segment at its best level for rows not yet scored
v13	0.17172	v5 with every segment at its best level for the public rows (Week 2 file)
v14	0.17032	v12's levels; buildings and contents ranked without this-year weather; motor spread x1.05
v15	not scored	v12 with the motor spread x1.05 only

B. Setting segment levels from public scores. For Tweedie p = 1.5, multiplying a segment's predictions by c changes the deviance by exactly

$$\Delta D = 4\left[(c^{-1/2}-1)A + (c^{1/2}-1)B\right], \qquad A = \sum y\,\mu^{-1/2}, \quad B = \sum \mu^{1/2},$$

so the best level is $c^* = A/B$, and any scored file that differs from another only by segment factors measures it. `11_joint_levels.py` fits one model, linear in the ten best levels, to 13 public scores (all within 0.0001). The public rows are a sample, so a segment's best level on them is off by $\mathrm{sd} = \sqrt{\phi/(\mathrm{share}\cdot\sum \mu^{1/2})}$, which the model fits as a separate term.

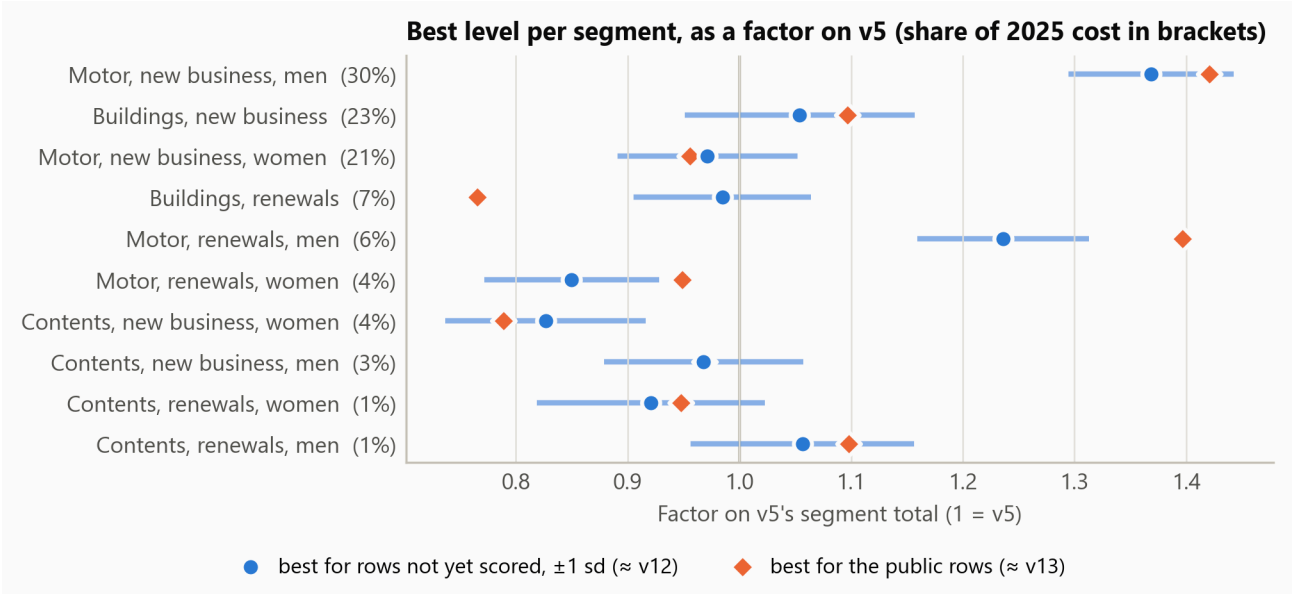


Figure 3: Best level per segment. v12 targets the blue points, v13 the orange ones. They differ most where the public rows say least.

Segment	Share of 2025 cost	Public-row luck sd	Best for public rows	Best for unseen rows	sd
Motor, new business, men	30.2%	8.3%	1.421	1.369	0.074
Buildings, new business	23.3%	12.8%	1.097	1.054	0.103
Motor, new business, women	21.5%	9.0%	0.956	0.971	0.081
Buildings, renewals	6.7%	23.5%	0.765	0.985	0.079
Motor, renewals, men	5.8%	16.8%	1.397	1.236	0.077
Motor, renewals, women	4.1%	18.2%	0.949	0.850	0.078
Contents, new business, women	3.8%	9.8%	0.789	0.826	0.090
Contents, new business, men	3.1%	10.2%	0.969	0.968	0.089
Contents, renewals, women	0.9%	18.6%	0.948	0.921	0.102
Contents, renewals, men	0.8%	19.5%	1.098	1.057	0.100

C. Week 2 forward test per line (from `2nd round/reports/05_model/holdout.csv`; D^2, fit 2023, score 2024).

Line	LightGBM on the developed prior	developed prior alone	LightGBM on Week 2 data only
All lines	0.1277	0.1259	0.1026
Motor	0.0842	0.0838	0.0801
Buildings	0.0754	0.0704	0.0122
Contents	0.0989	0.0985	0.0757

D. Files and how to reproduce. The two prediction files, their checksums and sources are in `final/1_predictions/manifest.csv`. The code and data are at https://github.com/IKSHRESTHA/IAA-Hackathon. `python final/pipeline.py` reruns the last stages of both pipelines from the intermediate results in the repository (about a minute) and checks the rebuilt files against the submitted ones; `--full` reruns every stage from the raw competition files. `final/README.md` lists the stages.

Reference. Zhou, H. et al. (2025). Memento: Fine-tuning LLM Agents without Fine-tuning LLMs. arXiv:2508.16153.